1ms Accumulator (accumulator_1ms)

Output Verification & Source Code

1. Objective

Design a module that accumulates a 4-bit signed input at a system clock frequency of 16.368 MHz, latching out a 16-bit signed accumulated total every 1ms. Since 16.368 MHz x 1ms = 16368 exactly, this corresponds to summing exactly 16368 samples per output update.

Overflow handling: per requirement, if the running total exceeds the 16-bit signed range (-32768 to +32767), the design wraps around (truncates) rather than saturating.

2. Output Verification

For this test run, a fixed input value of **data_in = 1** was applied on every clock tick. The expected result per 1ms window is therefore 1 x 16368 = 16368, which fits within the 16-bit signed range and does not overflow.

2.1 Simulation Transcript

```
=========================================================
 accumulator_1ms demo: fixed data_in = 1, every tick
 Ticks per 1ms window = 16368
 Expected result each window = 16368 (1 x 16368, wrapped in 16 bits)
=========================================================
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_accumulator_1ms_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
run -all
window 1 : result = 0011111111110000 (16368)   expected = 16368      MATCH
window 2 : result = 0011111111110000 (16368)   expected = 16368      MATCH
window 3 : result = 0011111111110000 (16368)   expected = 16368      MATCH
=========================================================
```

Figure 1: Simulation transcript, data_in = 1, 3 windows shown.

2.2 Waveform

The waveform below shows clk, reset, data_in, and result around the first 1ms window boundary. The result signal jumps from 0 to 0011111111110000 (16368) exactly at the transition.

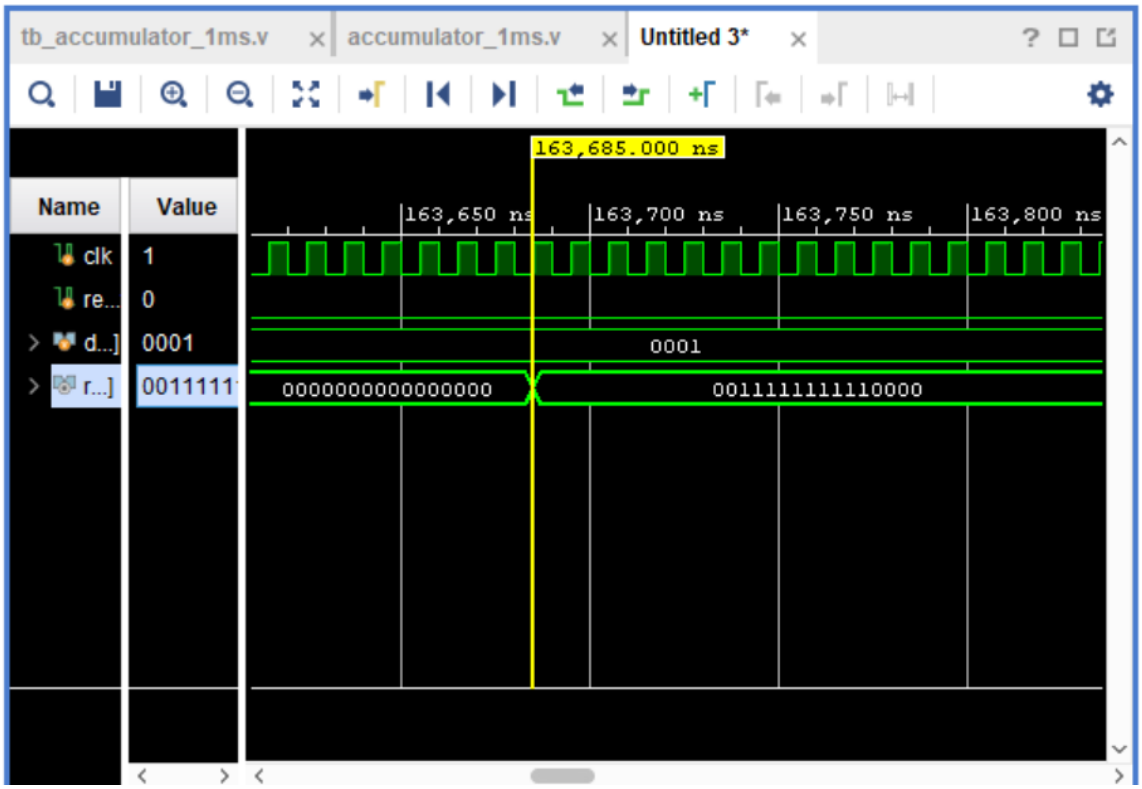

Figure 2: Waveform showing the result latch at the 1ms window boundary.

Note on timing: the simulation clock used here has a 10ns period (a fast, convenient period for simulation), not the real 16.368 MHz period (~61.0975ns). Because the testbench waits for exactly 16368 clock edges between updates (repeat (TICKS_PER_MS) @(posedge clk)), the design is correct by construction regardless of the simulation clock's period. To see the window boundary land exactly on real 1ms marks (1,000,000 ns) in the waveform directly, the testbench's clock generation can be changed to the true 16.368 MHz period.

2.3 Result Summary

Window	Result (binary)	Result (decimal)	Expected	Match
1	0011111111110000	16368	16368	PASS
2	0011111111110000	16368	16368	PASS
3	0011111111110000	16368	16368	PASS

All 3 windows matched the expected value of 16368 exactly, confirming the accumulator correctly sums 16368 samples per 1ms window with no overflow for this input value.

3. Source Code

accumulator_1ms.v

Purpose: Accumulates a 4-bit signed input every clock tick; every 16368 ticks (1ms at 16.368 MHz), latches the running total to a 16-bit signed output and resets the internal accumulator to 0.

```
// ================================================================
// Module: accumulator_1ms
// Description: Accumulates a 4-bit signed input on every clock edge.
//              At 16.368 MHz, exactly 16368 clock ticks occur in 1ms
//              (16.368 MHz x 1ms = 16368). Every 16368 ticks, the
//              running total is latched out to a 16-bit signed
//              output register, and the internal accumulator resets
//              to 0 to start the next 1ms window.
//
//              If the running total exceeds the 16-bit signed range
//              (-32768 to +32767), it wraps around (truncates) rather
//              than saturating -- per requirement.
// ================================================================
module accumulator_1ms #(
    parameter TICKS_PER_MS = 16368
)(
    input  wire              clk,        // 16.368 MHz system clock
    input  wire              reset,      // synchronous reset
    input  wire signed [3:0] data_in,    // 4-bit signed sample, new value every tick
    output reg  signed [15:0] result     // 16-bit signed accumulated total, updated every 1ms
);

    reg signed [15:0] acc;         // running total for the current 1ms window
    reg        [14:0] tick_count;  // counts 0 .. TICKS_PER_MS-1 (15 bits covers up to 32767)

    always @(posedge clk) begin
        if (reset) begin
            acc        <= 16'sd0;
            tick_count <= 15'd0;
            result     <= 16'sd0;
        end else begin
            if (tick_count == TICKS_PER_MS - 1) begin
                // last tick of this 1ms window: fold in this sample,
                // latch the total out, and start the next window at 0
                result     <= acc + data_in;   // wraps automatically if it overflows 16 bits
                acc        <= 16'sd0;
                tick_count <= 15'd0;
            end else begin
                acc        <= acc + data_in;
                tick_count <= tick_count + 1'b1;
            end
        end
    end

endmodule
```

tb_accumulator_1ms.v

Purpose: Demo testbench; applies a fixed data_in value (1) on every tick, waits exactly 16368 clock edges per window, and compares the latched result against the pre-calculated expected value across 3 consecutive 1ms windows.

```
`timescale 1ns/1ps
// ============================================================
// Demo testbench for accumulator_1ms
// Uses ONE fixed input value the whole time, so the expected
// result for every 1ms window is the same and can be
// pre-calculated: 1 x 16368 = 16368, which fits within the
```

```
// 16-bit signed range (-32768..+32767), so no overflow occurs
// for this value.
// ============================================================
module tb_accumulator_1ms;

    localparam TICKS_PER_MS = 16368;

    reg clk;
    reg reset;
    reg signed [3:0] data_in;
    wire signed [15:0] result;

    integer w;
    reg signed [15:0] expected;

    accumulator_1ms #(
        .TICKS_PER_MS(TICKS_PER_MS)
    ) DUT (
        .clk(clk),
        .reset(reset),
        .data_in(data_in),
        .result(result)
    );

    always #5 clk = ~clk;   // simulation clock (period value itself doesn't matter --
                             // what matters is counting exactly TICKS_PER_MS edges per window)

    initial begin
        clk      = 0;
        reset    = 1;
        data_in  = 1;              // <-- fixed value used every single tick
        expected = 16368;          // pre-calculated: (1 * 16368), fits in 16-bit signed, no wrapar
ound

        @(posedge clk);
        @(posedge clk);
        reset = 0;

        $display("============================================================");
        $display(" accumulator_1ms demo: fixed data_in = 1, every tick");
        $display(" Ticks per 1ms window = %0d", TICKS_PER_MS);
        $display(" Expected result each window = %0d (1 x %0d, wrapped in 16 bits)",
                  expected, TICKS_PER_MS);
        $display("============================================================");

        for (w = 1; w <= 3; w = w + 1) begin
            repeat (TICKS_PER_MS) @(posedge clk);
            #1;
            $display("window %0d : result = %b (%0d)   expected = %0d   %s",
                       w, result, result, expected,
                       (result === expected) ? "MATCH" : "MISMATCH");
        end

        $display("============================================================");
        $stop;
    end

endmodule
```